Missing Middle-Zoning in Toronto: Affordability Under Alternative Policies

Devon Kisob (21026216), Chanuth Weeraratna (20949079), Kevin Kim (20960485)

1 Project Scope

1.1 Updated Changes

Revised Problem Statement:

For the City of Toronto, how do alternative missing-middle zoning implementations change the evolution of housing affordability across census tracts over a 10-year horizon, under infrastructure capacity constraints and uncertain market response?

Revised Modelling Unit:

We model Census Tracts (CTs) within the City of Toronto as the primary agents in the simulation. CTs are geographic units defined by Statistics Canada to represent neighbourhoods with relatively homogeneous socioeconomic characteristics, typically containing between 2,500 and 8,000 residents. Toronto contains approximately ~500-600 CTs, making them an appropriate spatial scale for modelling neighbourhood-level housing dynamics.

Using CTs provides several advantages:

- Zoning regulations and infrastructure constraints operate at geographic scales similar to neighbourhoods rather than individual households
- Housing affordability, supply, and demographic characteristics vary significantly across neighbourhoods
- CT-level demographic and housing data are publicly available and widely used in housing and urban planning research [1]

Each CT agent will be initialized using data from the 2021 Statistics Canada Census, including:

- Median household income
- Number of households
- Renter and owner composition (share of households that rent vs own their homes)
- Housing stock counts

This allows each agent to represent a realistic neighbourhood with empirically grounded characteristics [2].

1.2 Output Metric

The primary output metric is a tract-level Housing Affordability Index (AI), computed separately for ownership and rental housing. The affordability index is the ratio of median household income to local housing costs, capturing the relative burden of housing expenses on households within each CT over time.

Ownership Affordability (for tract *CT* at time *t*):

$$AI_{own_{ct}} = \frac{MedianIncome_{ct}}{HomePrice_{ct,\,t}}$$

Rental Affordability (for tract *CT* at time *t*):

$$AI_{rent_{ct}} = \frac{MedianIncome_{ct}}{AnnualRent_{ct,\,t}}$$

Higher AI values indicate more affordable housing outcomes, while lower values indicate reduced affordability. This metric allows for a direct comparison of housing affordability across different neighbourhoods and policy scenarios. In the primary simulation output, time is on the x-axis and the AI on the y-axis. The trends of affordability can be evaluated by comparing the AI trajectories under alternative zoning scenarios (on both aggregated city-wide and disaggregated across CTs).

We will primarily track:

- The mean AI across CTs (AI_t) over time
- The distribution of affordability indices across CTs
- Spatial maps (ex. heatmap) of $AI_{ct,\,t}$ to highlight spatial-level inequality

1.3 Scenarios

S0 - Baseline (Status Quo Zoning)

This scenario represents current zoning conditions in the City of Toronto, under which large portions of residential land remain restricted to low-density housing forms. Missing-middle development is limited, and new housing supply is primarily concentrated in areas that were already zoned for higher-density construction.

S1 - City-Wide Missing-Middle Zoning

This scenario represents a possible city-wide legislation allowing for missing-middle housing forms (such as duplexes, multiplexes, etc.) across residential neighbourhoods. Toronto recently

legalized multiplexes city-wide in some forms already [3]. The goal is to increase housing supply across neighbourhoods, while the risk is a potential infrastructure strain on the city.

S2 - Targeted Missing-Middle Near Transit

This scenario represents a targeted zoning strategy in which missing-middle housing is permitted primarily in neighbourhoods near major transit hubs, which is a major planning strategy [4]. The goal is to concentrate the housing stock growth near transit corridors where the infrastructure capacity already exists, while the risk is uneven affordability gains and spatial inequality across the city.

S3 - Incentive-Based Missing-Middle Reform

This scenario represents zoning reforms combined with development incentives, such as reduced development charges or relaxed parking minimums, intended to lower the barriers to construction. The goal is to encourage development by reducing the overall regulatory barrier, while the risk is reduced municipal revenue.

Required Data

Across scenarios, zoning and incentive policies modify zoning eligibility, development probability, infrastructure strain, and housing demand at the CT level. Relative to the baseline (S0), S1 increases eligibility and development probability across most tracts, while S2 increases these variables only in a subset of tracts, concentrating growth spatially (near transit hubs). S3 further increases the development probability by reducing the regulatory barriers, but may increase infrastructure strain due to the reduced municipal revenue resulting from incentives. Data such as housing prices, rents, and affordability are all dependent variables that are subject to fluctuations in response to these changes. Thus, the simulation must rely on present CT-level demographic and housing data, historical housing data, and proxies to represent infrastructure capacity.

2 Conceptual Workflow

2.1 Agents

Table 1 summarizes the agents and models (pseudo-agents) included in the simulation, along with their attributes and behaviours. CTs are modelled as the primary agents, representing neighbourhood-level dynamics, while the supporting models capture bigger picture decisions/conditionals such as policy, demand, development, and infrastructure.

Table 1: Agents' Attributes and Behaviours

Agent	**Attributes [units]**	**Behaviours**
Census Tract (CT) Agent	• Median household income [$/year] • Number of households [count] • Housing units (rental) [units] • Housing units (ownership) [units] • Vacancy rate [%] • Average home price [$] • Average annual rent [$/year] • Demand pressure [index] • Infrastructure strain [index]	• Receives allocated housing demand • Evaluates the probability of new development • Updates the housing supply when development occurs • Updates vacancy, prices, and rents • Accumulates infrastructure strain over time
Policy Model (Pseudo-Agent)	• Zoning eligibility [binary] • Incentive level [index] • Scenario type [categorical]	• Applies zoning rules across the CTs • Modifies development conditions by scenario
Demand Allocation Model (Pseudo-Agent)	• Total housing demand [households/year] • Attractiveness weights [index]	• Distributes housing demand across CTs • Updates demand pressure in each CT
Development Model (Pseudo-Agent)	• Development probability [%] • Development size [units]	• Evaluates whether development occurs • Determines numbers of units added
Infrastructure Model (Pseudo-Agent)	• Infrastructure capacity [index] • Growth rate [%]	• Updates infrastructure strain based on housing growth • Feeds back into development constraints

2.2 Conceptual Agent-Based Workflow

Figure 1 represents the conceptual agent-based workflow for our simulation. In this framework, city-level housing demand growth is treated as exogenous, while development, prices, rent, vacancy, and infrastructure strain evolve endogenously within each CT. Zoning rules and incentive levels are fixed within a scenario, but vary across scenarios. The policy model defines zoning and incentive parameters based on our scenarios, which are fed into the demand model. Development decisions translate demand and infrastructure constraints into changes in the housing supply, which update neighbourhood-level prices, rents, and density within the CT agents. This behaviour is empirically calibrated using observed CT-level housing growth between 2016 and 2021, ensuring that simulated development probabilities and magnitudes reflect historical development patterns. Increased density contributes to infrastructure strain, creating a feedback loop into future decisions. Finally, affordability is computed as an output metric based on the CT characteristics, which is then stored for data analysis. Stochastic variation is introduced through demand growth and development occurrence, enabling Monte Carlo simulation to capture uncertainty in market responses and policy outcomes.

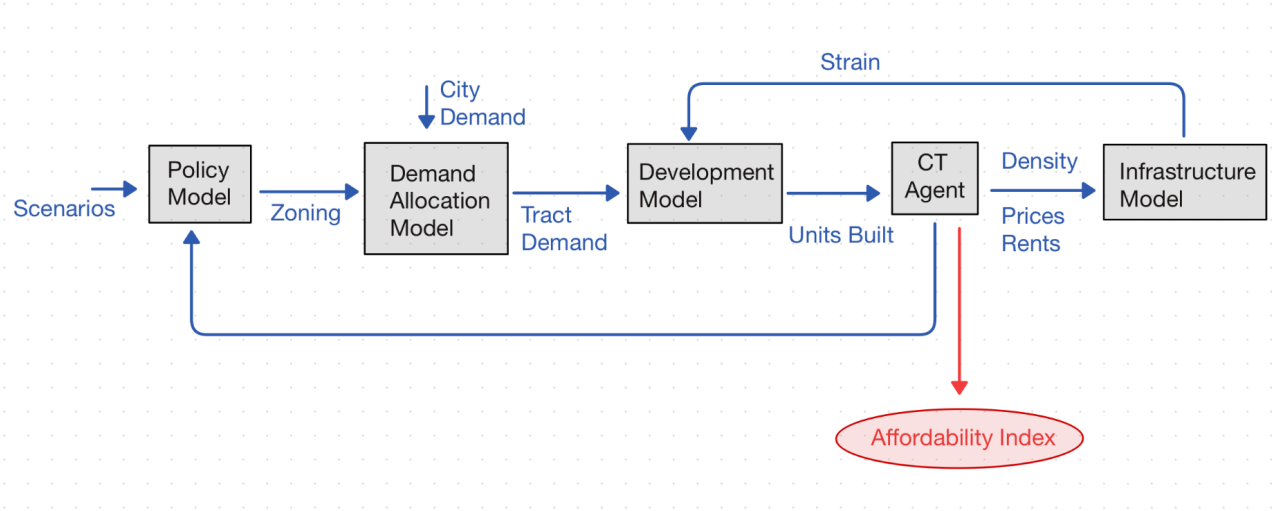

Figure 1: Agent-based Workflow for Project

3 Computational Workflow

3.0 Pre-simulation Calibration of Development Dynamics

(Note: This is not per time step; this loads data and trains the ML models before the simulation)

Evaluate Calibration Pipeline:

- **Check** load 2016 CT census table (units_2016 + covariates)
- **Check** load 2021 CT census table (units_2021 + covariates)
- **Check** join tables by CTUID
- **Update** delta_units = units_2021 − units_2016

Evaluate Zoning Feature Builder:

- **Check** load zoning/proxy table (by CTUID or spatial join)
- **Check** load transit proximity indicator (by CTUID)
- **Update** feature matrix X_calib for each CT

Evaluate Stage 1 Target Builder:

- **Check** development threshold τ (e.g., 10 net units OR 1% growth)
- **Update** y_stage1 = 1 if delta_units > τ else 0

Evaluate Stage 2 Target Builder:

- **Check** filter CTs where y_stage1 == 1
- **Update** y_stage2 = delta_units for filtered CTs

Evaluate ML Model Training:

- **Evaluate** Stage1_Model (logistic regression or random forest)
  - **Check** fit(X_calib, y_stage1)
  - **Update** fitted Stage1_Model
- **Evaluate** Stage2_Model (linear regression)
  - **Check** fit(X_calib[y_stage1==1], y_stage2)
  - **Update** fitted Stage2_Model

3.1 Simulation Loop

```
For each scenario s in {S0, S1, S2, S3}:
  Evaluate Policy Model:
    Check scenario type s
    Update zoning_rule parameters (e.g., eligibility mask by CTUID)
    Update incentive_level (e.g., development cost multiplier / probability boost)
    Update transit targeting rule (if scenario is targeted)
    Check g_cap(s) = max(0, g_base - λ * incentive_level(s))
    Update g_cap(s)


 For each realization r in {1..R}:
   Evaluate Initialization Model:
     Check load CT baseline attributes from 2021 census
     Update create CT_Agents list with per-CT attributes
     Update initialize home_price and annual_rent (placeholder/proxy if needed)
     Update initialize vacancy_rate (placeholder/proxy if needed)
     Update initialize infrastructure_capacity (proxy)
     Update set baseline_units = units_total at t=0

  For each time step t in {1..T} (1 year):

    Evaluate Demand Allocation Model:
      Check city_demand(t) (exogenous + stochastic shock)
      For each CT Agent i:
        Check attractiveness_i using current price/rent
      Update demand_pressure_i = allocate(city_demand, attractiveness_i)

    Evaluate Development Model: (hybrid ML call happens here)
      For each CT Agent i:
        Evaluate Policy Model (local application):
          Check zoning_eligible_i from scenario zoning rule
          Update zoning_eligible_i

        Check construct feature vector X_i,t including:
          - zoning_eligible_i
          - incentive_level
          - demand_pressure_i
          - current density proxy (units / households)
          - strain_i
          - transit_indicator_i

        Check Stage1_Model predicts p_dev_i,t = P(development | X_i,t)
         Update p_dev_i,t
```

Check sample dev_event_i,t ~ Bernoulli(p_dev_i,t)
Update dev_event_i,t

If dev_event_i,t == True:
Check Stage2_Model predicts units_added_i,t = E[delta_units | development, X_i,t]
Update units_added_i,t (apply max(0, …) and rounding)
Update CT.units_total += units_added_i,t
Update CT.units_rent and CT.units_own using tenure split rule (e.g., proportional to renter_share)

Evaluate CT Agent Market Update:
For each CT Agent i:
Check update vacancy_rate based on (demand_pressure_i, units_total_i)
Update vacancy_rate_i
Check update home_price_i,t using vacancy_rate and demand_pressure (placeholder price rule)
Update home_price_i
Check update annual_rent_i,t using vacancy_rate and demand_pressure (placeholder rent rule)
Update annual_rent_i

Evaluate Infrastructure Model (post-dev update):
For each CT Agent i:
Check update density proxy (units_total / households)
Update density_i
Check infra_load_i = ω0 + ω1 · density_i
Update infra_load_i
Check infra_capacity_i = infra_capacity_i · (1 + g_cap(s))
Update infra_capacity_i
Check strain_i = infra_load_i / infra_capacity_i
Update strain_i

Evaluate Output Model:
For each CT Agent i:
Check AI_own_i,t = income_i / home_price_i,t
Update store AI_own_i,t
Check AI_rent_i,t = income_i / annual_rent_i,t
Update store AI_rent_i,t
Check compute city-wide mean and quantiles across CTs
Update store aggregated metrics for scenario s, run r, time t

4 Computational Implementation

```python
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple, Any
import numpy as np

# ----------------------------
# Core Agent: Census Tract
# ----------------------------
@dataclass
class CensusTractAgent:
    ctuid: str

    # Empirical initialization targets (from Census where possible)
    median_income: float              # $/year
    households: int                   # count
    renter_share: float               # 0-1
    transit_indicator: float          # 0-1

    # Dynamic housing state
    units_total: int                  # units
    units_rent: int                   # units
    units_own: int                    # units
    vacancy_rate: float               # 0-1
    home_price: float                 # $
    annual_rent: float                # $/year

    # Demand + infrastructure state
    demand_pressure: float = 0.0      # index / households/year (allocated)
    infra_capacity: float = 1.0       # proxy index (placeholder)
    infra_load: float = 0.0           # proxy index (placeholder)
    strain: float = 0.0               # infra_load / infra_capacity

    # Baseline for strain proxy (helps defensibility)
    baseline_units: int = field(init=False)

    def __post_init__(self):
        self.baseline_units = max(self.units_total, 1)

    def update_infrastructure(self, omega0: float, omega1: float) -> None:
        """
        Placeholder: infra_load is linear in density proxy.
```

```
        Next deliverable: replace with better proxy and/or calibrate omega
parameters.
        """
        density_proxy = self.units_total / max(self.households, 1)
        self.infra_load = omega0 + omega1 * density_proxy
        self.strain = self.infra_load / max(self.infra_capacity, 1e-6)

    def apply_development(self, units_added: int) -> None:
        """
        Updates housing stock with a simple tenure split rule.
        Next deliverable: use empirical tenure split or scenario-specific
tenure assumptions.
        """
        units_added = int(max(0, round(units_added)))
        if units_added == 0:
            return

        rent_added = int(round(units_added * np.clip(self.renter_share,
0.2, 0.8)))
        own_added = units_added - rent_added

        self.units_total += units_added
        self.units_rent += rent_added
        self.units_own += own_added

    def update_market(self, price_kappa: float, rent_kappa: float, v_star:
float) -> None:
        """
        Placeholder: prices/rents respond to vacancy gap and demand
pressure.
        Next deliverable: calibrate to real price/rent dynamics or use
empirically justified elasticity ranges.
        """
        # Very simple vacancy update: vacancy falls when demand is high
relative to units.
        demand_per_unit = self.demand_pressure / max(self.units_total, 1)
        self.vacancy_rate = float(np.clip(self.vacancy_rate + 0.02 - 0.5 *
demand_per_unit, 0.0, 0.25))

        # Price/rent update
        self.home_price *= (1.0 + price_kappa * (v_star -
self.vacancy_rate))
        self.annual_rent *= (1.0 + rent_kappa * (v_star -
```

```
self.vacancy_rate))

    def affordability_own(self) -> float:
        return self.median_income / max(self.home_price, 1.0)

    def affordability_rent(self) -> float:
        return self.median_income / max(self.annual_rent, 1.0)

# ----------------------------
# Policy Model (Pseudo-Agent)
# ----------------------------
@dataclass
class PolicyModel:
    scenario_name: str
    incentive_level: float  # e.g., 0.0 baseline, >0 incentive
    eligible_ctuids: set    # set of ctuid eligible under zoning rule

    def is_eligible(self, ctuid: str) -> bool:
        return ctuid in self.eligible_ctuids

# ----------------------------
# Demand Allocation Model (Pseudo-Agent)
# ----------------------------
@dataclass
class DemandAllocationModel:
    base_city_demand: float      # households/year (placeholder)
    demand_growth: float         # per-year growth rate (placeholder)
    rng: np.random.Generator

    def city_demand(self, t: int) -> float:
        """
        Placeholder: deterministic growth + noise.
        Next deliverable: ground in Toronto population/household formation
estimates.
        """
        shock = self.rng.lognormal(mean=0.0, sigma=0.1)
        return self.base_city_demand * ((1 + self.demand_growth) ** t) *
shock

    def allocate(self, cts: List[CensusTractAgent], t: int) -> None:
        """
        Placeholder attractiveness: cheaper CTs are more attractive.
        Next deliverable: include transit, strain, and affordability-based
```

```
attractiveness.
        """
        total = self.city_demand(t)
        attractiveness = np.array([1.0 / max(ct.home_price, 1.0) for ct in
cts], dtype=float)
        weights = attractiveness / max(attractiveness.sum(), 1e-9)

        for ct, w in zip(cts, weights):
            ct.demand_pressure = float(total * w)

# ----------------------------
# Development Model (Pseudo-Agent) with Hybrid ML
# ----------------------------
@dataclass
class DevelopmentModel:
    stage1_model: Any  # sklearn-like classifier with predict_proba
    stage2_model: Any  # sklearn-like regressor with predict
    rng: np.random.Generator

    def feature_vector(self, ct: CensusTractAgent, policy: PolicyModel) ->
np.ndarray:
        """
        Minimal feature vector; expand later.
        Keep consistent between training and simulation.
        """
        zoning_eligible = 1 if policy.is_eligible(ct.ctuid) else 0
        density_proxy = ct.units_total / max(ct.households, 1)
        return np.array([zoning_eligible, policy.incentive_level,
ct.demand_pressure, density_proxy, ct.strain, ct.transit_indicator],
dtype=float).reshape(1, -1)

    def step(self, ct: CensusTractAgent, policy: PolicyModel) -> None:
        """
        Evaluate development occurrence and size using fitted models.
        """
        X = self.feature_vector(ct, policy)

        # Stage 1: probability of development
        p_dev = float(self.stage1_model.predict_proba(X)[0, 1])
        dev_event = self.rng.random() < p_dev

        if not dev_event:
            return
```

```
        # Stage 2: conditional development size
        units_added = float(self.stage2_model.predict(X)[0])

        # Apply units
        ct.apply_development(units_added)

# -----------------------------
# Infrastructure Model (Pseudo-Agent)
# -----------------------------
@dataclass
class InfrastructureModel:
    omega0: float
    omega1: float
    g_base: float            # baseline annual infra capacity growth (e.g.,
0.01)
    lambda_incent: float     # penalty per incentive unit (e.g., 0.005)

    def step(self, ct, policy) -> None:
        """
        Updates infrastructure load and capacity, then recomputes strain.
        Incentives reduce the growth rate of infrastructure capacity (proxy
for reduced municipal revenue).
        """
        # Load from density proxy
        density_proxy = ct.units_total / max(ct.households, 1)
        ct.infra_load = self.omega0 + self.omega1 * density_proxy

        # Capacity growth with incentive penalty
        g_cap = max(0.0, self.g_base - self.lambda_incent *
policy.incentive_level)
        ct.infra_capacity *= (1.0 + g_cap)

        # Strain
        ct.strain = ct.infra_load / max(ct.infra_capacity, 1e-6)

# -----------------------------
# Simulation Orchestrator
# -----------------------------
@dataclass
class Simulation:
    policy: PolicyModel
    demand_model: DemandAllocationModel
```

```
    dev_model: DevelopmentModel
    infra_model: InfrastructureModel

    price_kappa: float
    rent_kappa: float
    v_star: float

    def run(self, cts: List[CensusTractAgent], T: int) -> Dict[str,
List[float]]:
        """
        Returns time series of aggregated metrics (mean affordability).
        """
        mean_ai_own = []
        mean_ai_rent = []

        for t in range(T):
            # Demand allocation
            self.demand_model.allocate(cts, t)

            # Development step
            for ct in cts:
                self.dev_model.step(ct, self.policy)

            # Post-development: market + infrastructure update
            for ct in cts:
                ct.update_market(self.price_kappa, self.rent_kappa,
self.v_star)
                self.infra_model.step(ct, self.policy)

            # Output
            mean_ai_own.append(float(np.mean([ct.affordability_own() for ct
in cts])))
            mean_ai_rent.append(float(np.mean([ct.affordability_rent() for
ct in cts])))

        return {"mean_ai_own": mean_ai_own, "mean_ai_rent": mean_ai_rent}
```

5 Plan for Next Steps

5.1 Division of Responsibilities

At this stage, the overall structure of the agent-based model is defined, but several components remain underdeveloped. The next phase will focus on strengthening empirical grounding, refining behavioural functions, implementing the full simulation loop, and conducting sensitivity analysis for the Final Report.

Devon Kisob

- Lead data acquisition and preprocessing (2016–2021 CT data, zoning proxies, transit proximity)
- Implement the demand allocation model and attractiveness index
- Assist in spatial visualization (maps and inequality measures)

Kevin Kim

- Implement the CensusTract class and core simulation loop
- Develop and calibrate the development probability function (functional or ML-based)
- Integrate infrastructure feedback into development logic
- Ensure Monte Carlo structure and computational efficiency

Chanuth Weeraratna

- Implement price and rent update functions
- Conduct baseline and scenario runs
  Lead sensitivity analysis and robustness testing
- Develop comparative figures for stakeholder presentation (baseline vs reform scenarios)

References

[1] “Census Tract Data – Canadian Census,” Thompson Rivers University Library Guides, 2026. [Online]. Available: https://libguides.tru.ca/censuscanada/censustract. [Accessed: Feb. 27, 2026].

[2] “Boundary Files – Census 2021,” Statistics Canada, 2025. [Online]. Available: https://www12.statcan.gc.ca/census-recensement/2021/geo/sip-pis/boundary-limites/index-eng.cfm. [Accessed: Feb. 27, 2026].

[3] “Expanding Housing Options,” City of Toronto, 2026. [Online]. Available: https://www.toronto.ca/city-government/planning-development/planning-studies-initiatives/expanding-housing-options/. [Accessed: Feb. 27, 2026].

[4] “How to implement transit-oriented development,” C40 Knowledge Hub, 2024. [Online]. Available: https://www.c40knowledgehub.org/s/article/How-to-implement-transit-oriented-development?language=en_US. [Accessed: Feb. 27, 2026].